

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..37

INHERITANCE

 Copy as Markdown

Inheritance

A `FutureCall` class can extend another and inherit its future-call methods. You can use this to share common logic across your own classes, or to run future calls that a `module` exposes to the projects that depend on it. A subclass keeps every method it inherits and can add new methods or override inherited ones.

Extend a class

When one `FutureCall` extends another, the generated code exposes both the inherited methods and the new ones.

```
import 'package:serverpod/serverpod.dart';

class Greeter extends FutureCall {
  Future<void> hello(Session session, String name) async {
    session.log('Hello $name');
  }
}

class MyGreeter extends Greeter {
  Future<void> bye(Session session, String name) async {
    session.log('Bye $name');
  }
}
```

The `Greeter` class exposes `hello`, and `MyGreeter` exposes both the inherited `hello` and its own `bye`.

Expose future calls from a module

Mark the parent `abstract` when you want it to define methods without being scheduled on its own. Serverpod generates no accessor for an abstract `FutureCall`, so the methods become available only to a concrete subclass.



```
import 'package:serverpod/serverpod.dart';

abstract class Greeter extends FutureCall {
  Future<void> hello(Session session, String name) async {
    session.log('Hello $name');
  }
}

class MyGreeter extends Greeter {}
```

Here `Greeter` is not scheduled directly, and `MyGreeter` exposes the inherited `hello`.

This is the pattern a module uses to hand future calls to the projects that depend on it. The module ships an abstract `Greeter`, and the consuming project defines a concrete subclass such as `MyGreeter`. The code is generated in the consuming project, so `pod.futureCalls` there exposes the module's `hello` method through the local subclass.

A concrete subclass can still add its own methods on top of the inherited ones:

```
class MyGreeter extends Greeter {
  Future<void> bye(Session session, String name) async {
    session.log('Bye $name');
  }
}
```

This `MyGreeter` exposes both `hello` and `bye`.

Override an inherited method

A subclass can override an inherited method to change its behavior. An override replaces the parent's implementation:

```
import 'package:serverpod/serverpod.dart';

abstract class Greeter extends FutureCall {
  Future<void> hello(Session session, String name) async {
    session.log('Hello $name');
  }
}

class ExcitedGreeter extends Greeter {
  @override
  Future<void> hello(Session session, String name) async {
    session.log('Hello $name!!!');
  }
}
```

Here `ExcitedGreeter` exposes a single `hello` that logs `Hello $name!!!`. To build on the parent's behavior instead of replacing it, call `super.hello(session, name)` from inside the override.

The override must keep a signature compatible with the base method, following Dart's own rules: you can add optional parameters, but you cannot add required parameters or change the return type.

Related

- [Future calls](#): defining and scheduling a call.
- [Modules](#): sharing code between projects.